#4 Since `s` is not the empty string, it can be broken into a *head* character, `c` (the first character of `s`) and a *tail*, the rest of the string.
#5 If there is no outgoing edge in `node` for the character `c`, it means that we cannot traverse the trie any further, so that `s` is not stored in the subtree rooted at `node`.
#6 Otherwise, we recursively search `tail` into the subtree referenced by `children[c]`.

Listing 6.3 shows the trie's API counterpart, which in turn will call the method in listing 6.2; we will omit these wrapper methods for the other operations, whenever they are as trivial as for `search`.

Listing 6.3 method Trie::search

```
function
search(s)
      #1
  if this.root == null then
                                    #2
    return false
  else
    return search(this.root, s)
                                              #3
```

#1 Method `search` takes a string key `s` to be searched; it returns `true` if `s` is stored in the trie, or `false` otherwise.
#2 Check if the trie's root is `null`: if so, no string is stored in the trie, and we can return `false`.
#3 Otherwise, we forward the call to the root's `search` method.